

PROJET-DSAI

Guide de demarrage complet

De zero a la generation d'arguments convaincants

AXE 1 - Analyse

AXE 2 - Detection

AXE 3 - Generation

Python 3.10+ | PyTorch | scikit-learn | Hydra | HuggingFace

Ordre d'execution recommande

1. Installation & Datasets
2. Axe 1 : train.py encoder=features
3. Axe 2 : train.py dataset=hc3 encoder=roberta
4. Axe 3 : finetune.py + generate.py

0. PREREQUIS & INSTALLATION

1 Prerequisites systeme

- Python 3.10 ou superieur (verifier : `python --version`)
- Git installe et configure
- 8 Go de RAM minimum (16 Go recommandes)
- GPU optionnel (necessaire uniquement pour RoBERTa / Mistral 7B)

2 Cloner le depot Git

Clonage et navigation

```
git clone https://github.com/votre-org/Projet-DSAI.git
cd Projet-DSAI
```

3 Creer et activer l'environnement virtuel

Windows :

Windows

```
python -m venv env
env\Scripts\activate
# Le prompt affiche maintenant : (env) ...
```

macOS / Linux :

macOS / Linux

```
python -m venv env
source env/bin/activate
```

NOTA Toujours activer le venv avant de lancer une commande. Le prompt doit afficher (env) au debut.

4 Installer les dependances

Installation

```
pip install --upgrade pip
pip install -r requirements.txt

# Dependances principales :
# numpy, pandas, scikit-learn, matplotlib
# nltk, gensim, sentence-transformers, transformers, torch
# hydra-core, omegaconf, convokit, datasets, fpdf2
```

5 Telecharger les datasets

Le projet utilise deux datasets principaux :

- WinningArgCorpus (WAC) pour les Axes 1 et 3
- Human ChatGPT Comparison Corpus (HC3) pour l'Axe 2

Source originelle WAC :

<https://convokit.cornell.edu/documentation/winning.html>

Source originelle HC3 :

<https://huggingface.co/datasets/Hello-SimpleAI/HC3>

Pour tout telecharger et formater correctement :

Telechargement automatique

```
# WinningArgCorpus (Axes 1 & 3)
python -X utf8 scripts/download_datasets.py --dataset wac

# HC3 - Human ChatGPT Comparison Corpus (Axe 2)
python -X utf8 scripts/download_datasets.py --dataset hc3

# Ou les deux d'un coup :
python -X utf8 scripts/download_datasets.py --all
```

NOTA Les datasets seront formattés et sauvegardés dans :

- datasets/WinningArgCorpus/WAC.csv (19 714 arguments)
- datasets/HC3/train.csv (17 112 questions)

Si le telechargement echoue, vous pouvez telecharger les sources manuellement et les placer dans ces chemins exacts.

AXE 1 - ANALYSE D'ARGUMENTS

Predire quels arguments convainquent l'auteur du post (WAC dataset)

L'Axe 1 entraine un SVM sur le WinningArgCorpus pour predire si un argument va convaincre l'auteur du post original (label success=1). Quatre encodeurs sont disponibles. Chaque run sauvegarde le modele SVM (.pkl) dans le dossier outputs/ pour reutilisation par l'Axe 3.

Approche A - Features stylistiques pairwise [RECOMMANDEE CPU]

Calcule ~50 features numeriques (nb mots, hedges, liens, lisibilite Flesch-Kincaid...) et compare paires (gagnant, perdant). La plus rapide sur CPU.

```
python train.py encoder=features

# Sortie attendue :
#   Train : ~7 000 exemples | Test : ~1 800 exemples
#   AUC final : 0.55 - 0.65
#   Modele -> outputs/.../axe1_svm_features_wac.pkl
```

Approche B - TF-IDF + SVM [Baseline rapide]

Representation sac-de-mots pondere. Bon point de comparaison baseline.

```
python train.py encoder=tfidf
# AUC attendu : ~0.58 - 0.64
```

Approche C - Word2Vec + SVM

Embeddings de mots (100d) entraines sur le corpus, moyenne par texte.

```
python train.py encoder=w2v
```

Approche D - RoBERTa + SVM [GPU recommande]

Embeddings contextuels pre-entraines (1 024 dimensions). Meilleure performance attendue mais tres lent sur CPU.

```
python train.py encoder=roberta
# ATTENTION : ~30-60 min sur CPU
```

NOTA ORDRE RECOMMANDE SUR CPU : features (< 5 min) -> tfidf (~3 min) -> w2v (~8 min) -> roberta (GPU de preference)

Sweep et visualisation

```
# Lancer plusieurs encodeurs en sequence
python train.py -m encoder=features,tfidf,w2v

# Visualisation PCA des representations
```

```
python evaluate.py encoder=features
python evaluate.py encoder=tfidf

# Afficher la config sans lancer
python train.py --cfg job
```

Sorties Axe 1

```
outputs/2026-06-16/15-30-00_features_wac/
train.log                <- rapport complet (AUC, classification report)
.hydra/config.yaml       <- config exacte du run
axel_svm_features_wac.pkl <- MODELE SVM (nécessaire pour Axe 3)
pca_visualization.png    <- si evaluate.py lance
```

AXE 2 - DETECTION DE TEXTES SYNTHETIQUES

Distinguer textes humains vs generes par ChatGPT (HC3 dataset)

L'Axe 2 entraine un SVM sur le dataset HC3 pour detecter si un texte a ete ecrit par un humain (label 0) ou par ChatGPT (label 1). Le pipeline est identique a l'Axe 1 - seul le dataset change.

NOTA Preamble : avoir telecharge HC3 :
python -X utf8 scripts/download_datasets.py --dataset hc3

Commandes (ordre recommande)

```
# Approche A : RoBERTa + SVM (meilleure performance)
python train.py dataset=hc3 encoder=roberta

# Approche B : TF-IDF + SVM (baseline rapide sur CPU)
python train.py dataset=hc3 encoder=tfidf

# Approche C : Word2Vec + SVM
python train.py dataset=hc3 encoder=w2v

# Sweep complet
python train.py -m dataset=hc3 encoder=tfidf,w2v,roberta
```

Visualisation PCA

```
# Visualiser la separation humain vs ChatGPT dans l'espace 2D
python evaluate.py dataset=hc3 encoder=roberta
```

Sorties Axe 2

```
outputs/2026-06-16/15-45-00_roberta_hc3/
  train.log
  axe1_svm_roberta_hc3.pkl      <- MODELE SVM Axe 2 (necessaire pour Axe 3)
  pca_visualization.png
```

AXE 3 - GENERATION D'ARGUMENTS CONVAINCANTS

Fine-tuning LLM + Best-of-N + Prompt Engineering

L'Axe 3 utilise les modeles des Axes 1 et 2 pour generer des arguments convaincants. Deux strategies : (A) Fine-tuning du LLM sur WAC + selection Best-of-N par le SVM Axe 1, et (B) Prompt Engineering derive des features Axe 1 (zero-shot, pas de fine-tuning).

NOTA PREREQUIS OBLIGATOIRES avant Axe 3 :

1. Axe 1 entraine : outputs/.../axe1_svm_features_wac.pkl
2. Axe 2 entraine : outputs/.../axe1_svm_roberta_hc3.pkl

Notez les chemins exacts de ces fichiers .pkl avant de continuer.

Etape 1 - Fine-tuner le LLM sur les arguments gagnants

On fine-tune GPT-2 (ou Mistral avec LoRA) en next-token prediction uniquement sur les arguments success=1 du WAC. Le modele apprend le style des arguments convaincants.

```
# GPT-2 small - fonctionne sur CPU (~2-3 heures)
python finetune.py llm=gpt2

# Mistral 7B avec LoRA - GPU requis
python finetune.py llm=mistral

# Sortie : dossier finetuned_gpt2/ contenant le modele
```

Etape 2a - Generation Best-of-N

Pour chaque post original (OP) : genere N arguments, les score avec le SVM Axe 1, retient le meilleur. La selection utilise la fonction de decision du SVM comme oracle.

```
# Remplacer les chemins par vos fichiers .pkl reels
python generate.py strategy=best_of_n
    axe1.model_path="outputs/2026-06-16/.../axe1_svm_features_wac.pkl"
    axe2.model_path="outputs/2026-06-16/.../axe1_svm_roberta_hc3.pkl"

# Avec le modele fine-tune specifique
python generate.py strategy=best_of_n llm.model_id=finetuned_gpt2/

# Changer le nombre de candidats (default : 10)
python generate.py strategy=best_of_n strategy.n_candidates=20
```

Etape 2b - Generation par Prompt Engineering (zero-shot)

Analyse les features Axe 1 les plus discriminantes (via les coefficients du SVM) et les traduit en instructions naturelles dans le prompt. Pas de fine-tuning requis.

```
python generate.py strategy=prompt_eng
    axe1.model_path="outputs/2026-06-16/.../axe1_svm_features_wac.pkl"
    axe2.model_path="outputs/2026-06-16/.../axe1_svm_roberta_hc3.pkl"
```

```
# Ex. d'instructions generees depuis Axe 1 :  
#   'Use at least 2 concrete examples'  
#   'Cite external sources with links'  
#   'Keep sentences under 20 words'
```

Sorties Axe 3

```
outputs/generation/2026-06-16/15-00_gpt2_best_of_n/  
  generate.log           <- logs de generation  
  generation_report.csv  <- rapport : axel_score, axe2_authenticity  
  
finetuned_gpt2/         <- modele GPT-2 fine-tune (reutilisable)
```

REFERENCE RAPIDE - Toutes les commandes dans l'ordre

INSTALLATION

```
git clone <url-du-repo> && cd Projet-DSAI
python -m venv env
env\Scripts\activate          # Windows
# source env/bin/activate     # macOS/Linux
pip install -r requirements.txt
python -X utf8 scripts/download_datasets.py --all
```

AXE 1 - Analyse d'arguments (WAC)

```
python train.py encoder=features      # Recommande CPU - ~5 min
python train.py encoder=tfidf         # Baseline - ~3 min
python train.py encoder=w2v           # ~8 min
python train.py encoder=roberta       # GPU recommande
python evaluate.py encoder=features    # Visualisation PCA
```

AXE 2 - Detection synthetique (HC3)

```
python train.py dataset=hc3 encoder=roberta # GPU recommande
python train.py dataset=hc3 encoder=tfidf    # Baseline CPU
python evaluate.py dataset=hc3 encoder=roberta # Visualisation
```

AXE 3 - Generation d'arguments

```
python finetune.py llm=gpt2           # Fine-tuning ~2-3h CPU
python generate.py strategy=best_of_n \
    axel.model_path="outputs/.../axel_svm_features_wac.pkl" \
    axe2.model_path="outputs/.../axel_svm_roberta_hc3.pkl"

python generate.py strategy=prompt_eng \
    axel.model_path="outputs/.../axel_svm_features_wac.pkl" \
    axe2.model_path="outputs/.../axel_svm_roberta_hc3.pkl"
```

HYDRA - Commandes utiles

```
python train.py --cfg job              # Voir config sans lancer
python train.py -m encoder=tfidf,w2v,features # Sweep multi-run
python train.py encoder=tfidf seed=99     # Override n'importe quelle valeur
```

STRUCTURE DU PROJET

```
Projet-DSAI/
  configs/          <- Configs Hydra YAML
  dataset/          <- wac.yaml | hc3.yaml | grid.yaml
  encoder/          <- tfidf | w2v | roberta | features
  model/            <- svm.yaml
  generation/       <- config_generate.yaml | llm/ | strategy/
  src/
```

```
data/          <- loaders & preprocessing
features/      <- features stylistiques
encoders/      <- TF-IDF, W2V, RoBERTa, Features
models/        <- SVM classifier (save/load)
generation/    <- finetuning, generator, best_of_n, evaluator
datasets/      <- donnees brutes (non versionnees)
outputs/       <- resultats Hydra (non versionnees)
train.py       <- entree entraînement
evaluate.py    <- visualisation PCA
finetune.py    <- fine-tuning LLM (Axe 3)
generate.py    <- generation d'arguments (Axe 3)
scripts/      <- download_datasets.py, generate_pdf.py
```